Position-Independent Detection of Voiced Consonants in Isolated Words Using Spectrum-Based Methods and an AI Baseline

Bunyod Suvonov and Giorgio Aman Soetomo

Abstract—This paper studies position-independent full-word detection of four voiced consonants, d, g, n, and r, using real recordings from Speech Commands v0.02. An onset-only formulation is too narrow for letter-presence detection, because the target consonant may appear at the beginning, middle, or end of the word. A 24-keyword benchmark is constructed so that the targets occur in initial, medial, and final positions. We compare five classical detectors and one AI detector: global FFT template matching, MFCC mean-template scanning, MFCC + dynamic time warping (DTW), local STFT mean-template scanning, local STFT k-nearest-neighbor (k-NN) scanning, and a small multilayer perceptron (MLP) operating on full-word log-Mel features. The FFT/STFT spectrum detectors are implemented in MATLAB, while the MFCC/DTW and MLP detectors provide literature and AI comparisons. On balanced speaker-independent test sets, the classical methods achieve macro F1 between 65.17% and 67.03%, while the AI detector reaches 86.39% macro F1 and 86.45% macro accuracy. The main conclusion is that coarse template detectors have high recall but poor precision under word-level labels and weak phoneme alignment, whereas the trained AI model learns a substantially sharper decision boundary for the full-word task.

I. Introduction

Speech is a time-domain waveform, but many phonetic cues become much clearer after short-time spectral analysis. Sampling, Fourier analysis, and local spectra make physically meaningful structure visible. This work uses those tools to describe voiced consonants and detect whether they are present in spoken isolated words.

The critical methodological issue is position. A detector that only examines the beginning of a word is valid for d in down, but not for d in bed or r in zero. Therefore, in this report we study position-independent detection of four voiced consonants:

- d: voiced alveolar stop,
- g: voiced velar stop,
- n: voiced nasal stop,
- r: voiced liquid.

The study is organized around three questions:

1) How can we extract local spectral templates for consonants when the dataset provides word labels but not phoneme boundaries?
2) How well do classical spectrum, MFCC, and DTW detectors work when the target consonant may occur anywhere in the word?
3) How much improvement is obtained by an AI detector trained directly on whole-word features?

Our main contribution is a corrected and fully position-independent comparison on real speech data. Instead of assuming all target consonants are word-initial, we construct local target templates from approximate phoneme-position priors and then scan the entire active part of each word at test time.

II. Related Work

Classical speech recognition relied heavily on short-time spectral analysis and template matching. Rabiner and Schafer describe the central role of short-time processing in speech analysis [1]. Davis and Mermelstein introduced MFCCs as a compact and effective spectral representation for speech recognition [2]. Sakoe and Chiba proposed DTW to align speech templates under speaking-rate variability [3]. These ideas motivate the literature-based baselines reproduced in this report.

Modern speech systems increasingly use learned models rather than direct template matching. Hinton et al. summarized the impact of deep neural networks on acoustic modeling [4]. For limited-vocabulary speech tasks, Sainath and Parada showed that compact neural networks can outperform conventional baselines in keyword spotting [5]. We use Speech Commands because it is a standard real dataset for limited-vocabulary speech recognition and keyword spotting [6].

Our comparison therefore includes MATLAB spectrum methods based on FFT and STFT, literature-style MFCC/DTW methods, and an AI detector trained on the same real speech recordings.

III. Problem Formulation and Theory

A. Sliding-Window Letter Detection

Let $x[n]$ be a word recording and let $[t_s, t_e]$ denote its active speech region after simple energy-based voice activity detection. We slide a fixed analysis window across that active region and obtain local segments

$$w_m[n] = x[n + t_m], \quad m = 1, \ldots, M. \tag{1}$$

For a target consonant c, a template bank $B_c = \{b_{c,1}, \ldots, b_{c,K_c}\}$ is built from training recordings. A generic scan detector uses

$$S_c(x) = \max_m \max_{b \in B_c} \mathrm{sim}(\phi(w_m), b), \tag{2}$$

where $\phi(\cdot)$ is the chosen local feature map and $\mathrm{sim}(\cdot,\cdot)$ is cosine similarity. The final decision is

$$\hat{y}_c(x) = \begin{cases} 1, & S_c(x) \geq \tau_c, \\ 0, & S_c(x) < \tau_c, \end{cases} \tag{3}$$

with threshold τ_c chosen on the validation set.

B. Approximate Phoneme-Position Prior

Speech Commands provides word labels but not phoneme boundaries. To initialize target templates, we map each selected keyword to a simple ARPABET pronunciation and approximate the target-phoneme center by a uniform partition of the active word. If a word has L phonemes and the target occurs at zero-based phoneme index i, we estimate its center as

$$\hat{t}_i = t_s + \frac{i+0.5}{L}(t_e - t_s). \tag{4}$$

This weak alignment assumption is used only during training to choose template windows. Word labels are also used to define positive and negative examples and to compute ground-truth evaluation metrics. They are not used as inputs to the detector at test time: the detector receives only the waveform-derived spectral features and then scans the whole active region.

C. Short-Time Spectrum, MFCC, and DTW

For a frame window $h[n]$, the STFT is

$$X_m[k] = \sum_{n=0}^{N-1} x[n+mH]h[n]e^{-j2\pi kn/N}. \tag{5}$$

The local STFT feature used in this study is the average log-magnitude spectrum over a 100 ms analysis window. This keeps the detector directly interpretable in the frequency domain.

MFCCs compress the short-time spectrum into a perceptually motivated cepstral representation. If $E_m[b]$ is the Mel-band energy of frame m and band b, then

$$c_m[q] = \sum_{b=0}^{B-1} \log E_m[b] \cos\left(\frac{\pi(b+1/2)q}{B}\right). \tag{6}$$

DTW then compares two MFCC sequences by the minimum dynamic-programming path cost under local step constraints [3]. In our detector, DTW is applied between every scanned test window and a small bank of representative target templates.

D. AI Detector

For the AI method, each full word is converted into a resized log-Mel spectrogram vector $z(x)$. We train one binary MLP per target consonant:

$$p_c(x) = \sigma(W_2\rho(W_1\tilde{z}(x) + b_1) + b_2), \tag{7}$$

where $\tilde{z}(x)$ is standardized, $\rho(\cdot)$ is ReLU, and $\sigma(\cdot)$ is the sigmoid function. This model sees the entire word at once and is not limited to one local template.

IV. Dataset and Experimental Setup

A. Real Data and Target Selection

We use the full Speech Commands v0.02 dataset [6]. From it, we select 24 keywords whose pronunciations cover the target consonants in different positions. To keep the computation comparable across all methods, each keyword is capped at 150 training recordings, 60 validation recordings, and 120 test recordings, giving 7,920 recordings in total. The split is speaker-independent: the hexadecimal speaker hash modulo 100 defines train, validation, and test partitions.

Table I shows the positive keywords for each target consonant. The labels I/M/F denote initial, medial, and final target positions in the approximate pronunciation.

For each target consonant, the word label is converted into a binary presence label using the pronunciation table. For example, dog is positive for d and g, but negative for n and r. The negative class is sampled from the remaining selected keywords that do not contain that target. The negative examples are subsampled to match the number of positive examples, so every detection problem is balanced. During testing, these labels are used only to score the predictions, not to generate them.

B. Assumptions and Preprocessing

The experiments use the following assumptions:

- Speech Commands provides word labels but not phoneme boundaries.
- Active speech can be isolated by an energy threshold on 20 ms frames with 10 ms hop.
- Approximate phoneme timing can be estimated by dividing the active region according to pronunciation length.
- Detection should be position-independent at test time, so all local methods scan the full active word rather than only the onset.

Each recording is mean-removed and peak-normalized. The active region is then detected by thresholding short-time energy at 10% of the utterance maximum. Local detectors use a 100 ms analysis window, a 25 ms STFT frame, 10 ms hop, and 512-point FFT. MFCC features use 26 Mel filters and 13 cepstral coefficients.

C. Implemented Methods

We compare six methods. The global FFT, local STFT mean-template, and local STFT k-NN detectors form the MATLAB spectrum-based implementation. MFCC mean-template and MFCC + DTW are literature baselines, and the MLP is the AI detector used to solve the same detection task and compare against the classical methods. Table II summarizes the models, assumptions, and decision rules.

TABLE I: Target consonants, phonetic roles, and positive keyword set used in the experiments

Target	Phonetic role	Positive keywords used for detection
d	voiced alveolar stop	backward(F), bed(F), bird(F), dog(I), down(I), forward(F)
g	voiced velar stop	dog(F), go(I)
n	voiced nasal stop	down(F), marvin(F), nine(I/F), no(I), on(F), one(F), seven(F)
r	voiced liquid	forward(M), four(F), marvin(M), right(I), three(M), tree(M), zero(M)

TABLE II: Implemented algorithms, feature spaces, assumptions, and decision rules

Method	Feature / model	Training assumptions and structure	Test-time decision rule
Global FFT template	Whole active-word log FFT spectrum	Build one positive mean template per letter from whole-word spectra; threshold tuned on validation set	Detect the letter if cosine similarity to the positive template exceeds the tuned threshold
MFCC mean-template scan	13-dimensional MFCC mean of each 100 ms local window	Use pronunciation-based target windows to form initial/medial/final positive template means	Scan the active word and take the maximum cosine similarity over all windows and templates
MFCC + DTW	Sequence of MFCC vectors inside each 100 ms local window	Use pronunciation-based target windows to form a representative positive bank; tune template count and Sakoe–Chiba band on validation data	Scan the active word and use the negative minimum DTW distance as the score
Local STFT mean-template scan	Average local log spectrum of each 100 ms window	Use pronunciation-based target windows to form initial/medial/final positive spectral templates	Scan the active word and take the maximum cosine similarity over all windows and templates
Local STFT k-NN scan	Bank of local average log spectra	Store representative target windows; tune k on validation data	Scan the active word and use the best top-k average cosine similarity as the score
AI MLP	Whole-word resized log-Mel spectrogram vector	Train one binary MLP per letter with ReLU, sigmoid output, Adam, and early stopping	Use the output probability and a validation-tuned threshold to decide letter presence

D. Pseudo-Code

The algorithms below summarize the implemented pipelines.

Algorithm 1: Common preprocessing

1) Read the waveform, subtract the mean, and divide by the peak magnitude.
2) Compute frame energies on 20 ms windows with 10 ms hop.
3) Mark the active region as the contiguous high-energy part of the word.
4) Generate overlapping 100 ms analysis windows across the active region.

Algorithm 2: Build classical target templates

1) For each positive training word, map the word to its phoneme sequence.
2) Estimate the target-phoneme center using the uniform phoneme-position prior in (4).
3) Select the scanned window whose center is closest to that estimated target position.
4) Extract the local STFT or MFCC feature and add it to the target template bank.

Algorithm 3: Full-word classical detection

1) Apply Algorithm 1 to the test recording.
2) For every scanned local window, compute the chosen feature.
3) Compare that feature with the target bank by cosine similarity or DTW.
4) Keep the best score over all windows and threshold it to obtain the final binary decision.

Algorithm 4: AI detection

1) Convert the entire active word into a resized log-Mel feature map.
2) Standardize features using training-set statistics.
3) Train one binary MLP for each target consonant.
4) Use the output probability and a validation-tuned threshold for test-time detection.

V. Spectral Analysis

Figure 1 shows the average local spectra extracted from the estimated target windows. Even with weak phoneme alignment, the targets remain distinguishable. The stop consonants d and g show different low-to-mid frequency patterns; n concentrates more low-frequency energy because of nasal resonance; and r has a smoother broadband transition.

Figure 2 is equally important for validating the corrected task definition. The highlighted windows are not confined to the beginning of the word: g is captured at the end of dog, n at the end of seven, and r in the middle of zero. This is the main methodological difference from an onset-only report.

VI. Results and Comparisons

A. Overall Comparison

Table III reports the average precision, recall, F1, and accuracy over the four balanced binary detection tasks.

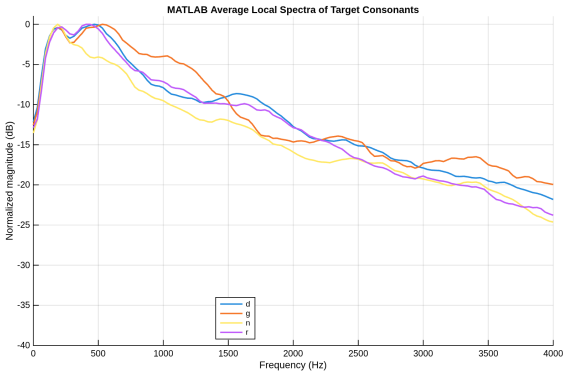

Fig. 1: MATLAB average local spectra of the four voiced consonants extracted from approximate target windows.

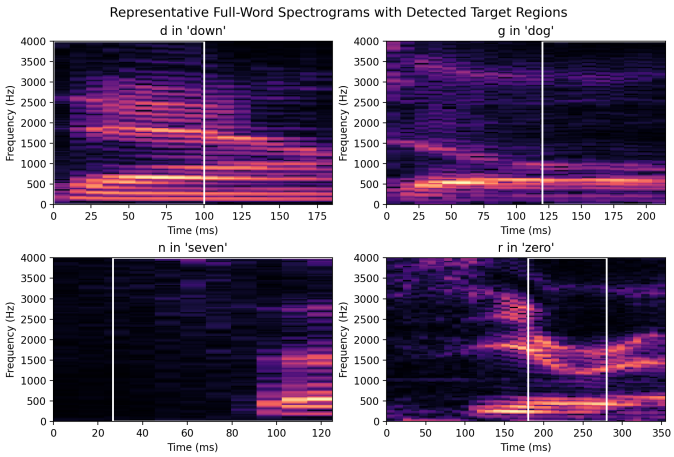

Fig. 2: Representative full-word spectrograms with highlighted target windows. The examples include initial, final, and medial consonant positions.

The main qualitative result is clear: the classical detectors are recall-heavy but imprecise, while the AI detector is balanced and substantially stronger.

The MATLAB spectrum implementation was also run separately for the FFT/STFT detectors. It gave comparable macro F1 values: 66.68% for global FFT, 66.55% for local STFT mean-template scanning, and 66.84% for local STFT k-NN scanning. These values support the same conclusion as Table III: classical spectrum scanning is useful for spectral interpretation, but it is not as selective as the trained AI detector under weak phoneme alignment.

Figure 3 presents the same comparison graphically. The five classical methods cluster closely together because they all depend on weakly aligned target templates. Once the target is allowed to occur anywhere in the word, coarse positive-template scanning becomes much less selective than in the earlier onset-only formulation.

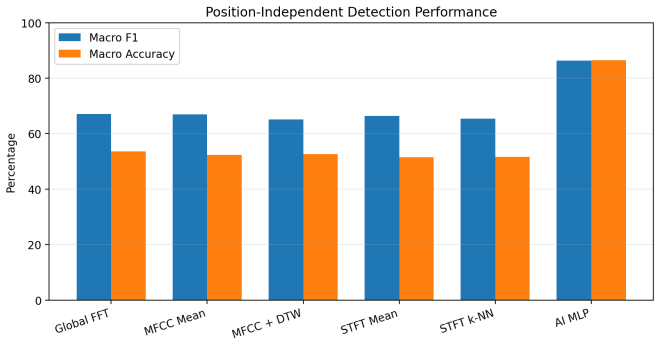

Fig. 3: Macro F1 and macro accuracy for all implemented methods.

Fig. 4: Per-letter detection F1 for a literature baseline, a local-spectrum baseline, and the AI detector.

B. Per-Letter Comparison

Figure 4 compares one literature-style baseline (MFCC + DTW), one local-spectrum classical detector (STFT k-NN), and the AI detector on each letter. The AI detector is best for all four consonants:

- d: 85.10% F1,
- g: 86.38% F1,
- n: 83.62% F1,
- r: 90.45% F1.

Among the classical methods, performance is especially limited for letters whose target windows are hard to isolate with only word-level supervision. This is expected: the classical detectors know how the target should look locally, but they do not learn a global representation of the entire word.

C. Discussion

The corrected experiments lead to four main conclusions.

1) Letter detection should be position-independent. A detector restricted to the word onset would miss valid cases such as d in bed, n in seven, or r in zero. The present report avoids that error by scanning the full active region.

TABLE III: Average detection performance across the four target consonants on the speaker-independent test split

Method	Macro precision (%)	Macro recall (%)	Macro F1 (%)	Macro accuracy (%)
Global FFT template	52.53	94.17	67.03	53.57
MFCC mean-template scan	51.25	96.52	66.93	52.30
MFCC + DTW	51.92	89.86	65.17	52.59
Local STFT mean-template scan	50.84	95.74	66.36	51.48
Local STFT k-NN scan	51.15	91.94	65.41	51.66
AI MLP	87.00	85.81	86.39	86.45

2) Weak phoneme alignment is the main bottleneck for classical detectors. The classical methods achieve recall around 90–97%, but precision only around 51–53%. They often find a spectrally similar local region somewhere in the word, even when the target consonant is absent. This high-recall/low-precision behavior is exactly what one should expect from positive-template scanning with only approximate target localization.

3) Literature baselines and spectrum baselines are both informative. MFCC mean-template and MFCC + DTW are the methods reproduced from the speech-recognition literature. Global FFT, local STFT mean-template, and local STFT k-NN are MATLAB spectrum methods based on direct Fourier analysis. In the full-word setting, none of the classical methods dominates decisively; all are constrained by the same lack of exact phoneme boundaries.

4) The AI detector is the most effective model in this study. The MLP sees the entire log-Mel representation of the word and learns nonlinear boundaries between positive and negative examples. It improves macro F1 from roughly 65–67% for the classical methods to 86.39%, and it is also much more balanced in precision and recall.

There are still limitations. The target timing is approximate rather than forced-aligned, and the g detector has only two positive keyword types (dog and go). A phoneme-aligned corpus would allow a cleaner study of purely acoustic consonant spotting. Even so, the current design is substantially better aligned with letter-presence detection than an onset-only formulation.

VII. Conclusion

We analyzed the spectra of four voiced consonants in MATLAB, implemented MATLAB spectrum-based detectors, reproduced literature-style MFCC/DTW baselines, and trained an AI detector on real speech data. The crucial formulation choice was to treat the task as position-independent letter detection inside words rather than onset-only classification.

Under that formulation, the five classical detectors all show similar high-recall/low-precision behavior, while the AI MLP achieves the strongest overall result with 86.39% macro F1 and 86.45% macro accuracy. The main scientific lesson is that Fourier-based spectrum tools remain useful for spectral interpretation and baseline detection, but once target timing becomes uncertain, learned whole-word representations are much more robust than weakly aligned local templates.

References

[1] L. R. Rabiner and R. W. Schafer, Digital Processing of Speech Signals. Englewood Cliffs, NJ: Prentice-Hall, 1978.

[2] S. B. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," IEEE Transactions on Acoustics, Speech, and Signal Processing, vol. 28, no. 4, pp. 357–366, 1980.

[3] H. Sakoe and S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," IEEE Transactions on Acoustics, Speech, and Signal Processing, vol. 26, no. 1, pp. 43–49, 1978.

[4] G. Hinton, L. Deng, D. Yu, G. E. Dahl, A.-r. Mohamed, N. Jaitly, A. Senior, V. Vanhoucke, P. Nguyen, T. N. Sainath, and B. Kingsbury, "Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups," IEEE Signal Processing Magazine, vol. 29, no. 6, pp. 82–97, 2012.

[5] T. N. Sainath and C. Parada, "Convolutional neural networks for small-footprint keyword spotting," in Proceedings of Interspeech, 2015, pp. 1478–1482.

[6] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," arXiv preprint arXiv:1804.03209, 2018.